프로세스와 자원의 하루

ls ⏎ — 그 한 줄 뒤에서 벌어지는 모든 일

오늘의 도착점

명령 한 줄이 fork → execve → 스케줄러 → 주소 변환 → wait 로 처리되는 전 과정을 빈칸 없이 이어 말한다

▶ 도구 모음 (index)

📋 Day 2 워크시트

모든 도구·워크시트: devops0906.github.io/AI_CAMPUS/os-day2/anim/

복습 다리

어제의 마지막 화면에서 오늘이 시작됩니다

Day 1 — 우리가 본 것

- File System — 모든 것은 파일
- GUI와 CLI는 같은 OS의 두 얼굴
- Terminal에서 명령을 직접 실행
- ls, cd, cat … 명령의 '겉모습'

Day 2 — 오늘 열어볼 것

- 그 명령은 '누가' 실행하는가 → Process
- CPU 차례는 누가 정하나 → 스케줄러
- Memory는 누가 나눠 주나 → buddy·paging
- 끝난 명령은 누가 거두나 → wait

오늘의 지도

여덟 세션 — 마지막 S8에서 전부 한 번씩 다시 만납니다

S1 프로세스의 일생
R·S·Z, 좀비

S2 fork()
복사기

S3 execve()·wait()
변신과 회수

S4 O(n)→O(1)
CPU 나누기 ①

S5 CFS
CPU 나누기 ②

S6 buddy·slab
Memory 배급

S7 가상 메모리
지도·문지기·회수

S8 종합
명령 한 줄의 여행

각 세션 = 도착점 → 애니메이션으로 관찰 → 워크시트에 답 → 확인

이 시간의 도착점

ps 화면의 상태 글자 R·S·Z를 읽고,
좀비가 왜 「Memory가 아니라 표의 한 줄」인지 설명한다

힌트 — 지금 이 순간에도 여러분 컴퓨터에는 수백 개의 프로세스가 있습니다. 대부분 무엇을 하고 있을까요?

프로세스 = 실행 중인 프로그램 + OS의 장부 한 줄

장부(프로세스 표)에 적히는 것

- PID — 주민번호
- PPID — 부모가 누구인가
- 상태 — R(실행/대기) · S(잠듦) · Z(좀비)
- 종료값 — 죽으며 남기는 숫자 한 개

상태의 진실

- 대부분은 S — 일이 올 때까지 잠들어 있다
- R은 '실행 중 + 실행하고 싶음'의 줄서기
- Z는 죽었는데 표에서 안 지워진 상태
- → 왜 안 지울까? 가 다음 슬라이드

좀비 — 시체가 아니라, 아직 안 찾아간 성적표

exit(7) 이후에 남는 것

Memory·CPU는 즉시 회수됩니다. 남는 것은 표 한 줄 — 「PID 1234, 종료값 7」.
부모가 wait()로 이 성적표를 찾아가야 표에서 지워집니다.

부모가 안 찾아가면

Z 상태로 표에 계속 남음 — 수천 개 쌓이면 표가 가득 찹니다.

부모가 먼저 죽으면

PID 1(systemd)이 입양해 대신 wait — 표를 청소합니다.

실습 1

프로세스의 일생 — 직접 관찰

할 일

① 생명주기 도구의 시나리오 ①→③을 순서대로 재생
② 각 단계에서 다음 상태를 먼저 예측
③ 워크시트 S1의 답 3개 채우기

결과물

워크시트 S1 완료 (상태 글자·좀비·PID 1)

▶ 프로세스 생명주기

📋 Day 2 워크시트

확인 「좀비가 왜 표의 한 줄인지, 옆 사람에게 30초로 설명을 마친 조는 손 들어 주세요.」

이 시간의 도착점

fork()가 CPU와 Memory를 어떻게 복제하는지,
그리고 반환값 한 칸이 어떻게 부모/자식의 길을 가르는지 설명한다

예측 — 함수를 '한 번' 호출했는데 return이 '두 번' 일어날 수 있을까요?

fork() = 프로세스 복사기

복제되는 것

- Memory 전체 — 코드·변수 그대로
- CPU 레지스터 — PC(다음 실행 위치)까지
- → 자식도 fork '직후'부터 시작
- 파일 핸들 등 장부 대부분

딱 하나 다른 것

- fork()의 반환값이 담기는 레지스터
- 부모 ← 자식의 PID (예: 1234)
- 자식 ← 0
- 이 한 칸이 if(pid==0)의 갈림길

복제 이후는 남남 — 독립 Memory의 증명

실험

fork 전에 x=10. 자식이 x=99로 바꾼다. → 부모의 x는?

답: 부모의 x = 10

복제된 순간부터 서로 다른 Memory — 한쪽의 수정이 넘어가지 않는다.

심화: Copy-on-Write

실제 Linux는 일단 같이 보다가, '쓰는 순간에만' 그 페이지를 복사한다.

실습 2

fork() — 복사와 갈림길

할 일

① fork 도구: 부모/자식 패널의 레지스터를 비교
② x=99 실험으로 독립 Memory 확인
③ 워크시트 S2의 답 4개 채우기

결과물

워크시트 S2 완료 (return 2회 · 자식 0 · x=10 · COW)

▶ fork() 해부

📋 Day 2 워크시트

확인「부모·자식 패널에서 단 하나 다른 레지스터가 무엇이었는지, 조별로 한 단어로 외쳐 주세요.」

이 시간의 도착점

Shell의 3박자 fork → execve → wait 를 완성하고,
「성공한 execve는 돌아오지 않는다」를 이유와 함께 말한다

예측 — execve("ls") 다음 줄에 print를 써 두면, 그 print는 실행될까요?

execve() = 변신 — 철거, 적재, 그리고 PC 재설정

① Memory 철거 — 기존 코드·데이터 전부 삭제

② 새 프로그램 적재 — ls의 코드가 들어옴

③ PC를 새 프로그램의 첫 줄로 — 되돌아갈 주소가 없다

그래서 다음 줄은 영원히 미실행

(execve가 '실패'했을 때만 -1로 돌아온다)

그래도 PID는 그대로

몸은 바뀌어도 장부의 줄은 같은 줄 — 같은 프로세스.

wait() = 회수 — 잠들었다, 성적표를 받고 깨어난다

흐름

부모 wait() → S로 잠듦 → 자식 exit(7) → 좀비(성적표 7) → OS가 부모를 깨움 → 성적표 회수, 표에서 삭제

이미 좀비가 있다면

wait는 잠들지 않고 즉시 성적표를 받아 온다.

3박자 완성

bash: fork(복제) → 자식 execve(변신) → 부모 wait(회수) → Prompt 복귀

실습 3

Shell의 3박자 완성

할 일

① execve 도구: 철거→적재→PC 재설정 관찰
② wait 도구: 잠듦→좀비→회수, 고아 입양까지
③ 워크시트 S3의 답 5개 채우기

결과물

워크시트 S3 완료 (3박자·PID 유지·성적표·systemd)

▶ execve() 해부 | ▶ wait() 해부 | 📋 Day 2 워크시트

확인 「3박자를 손가락 세 개로 꼽으며 10초 안에 말해 보세요. 셋 다 나온 조는 손!」

이 시간의 도착점

「전부 훑기」의 비용을 숫자로 말하고, 비트맵과 배열 교체가 그 비용을 상수(연산 2회)로 만든 원리를 설명한다

예측 — task가 1,000개면, 다음 차례 한 번 고르는 데 비교가 몇 번 필요할까요?

O(n) — 정직하지만 비싼 방법

동작

- 매번 모든 task를 훑는다
- 점수 goodness = counter + priority
- 최고 점수가 다음 주인공
- epoch: 전원 소진 시 counter 재충전

문제

- task 1,000개 → 비교 1,000회
- 선택할 때마다, 매번
- task가 늘수록 OS 자신이 느려짐
- 서버 시대에 치명적

O(1) — 훑지 않고 '계산'한다

두 가지 발명

① 우선순위 140줄 + 비트맵: 최고 우선순위 줄 찾기 = find-first-bit 1회
② active / expired 두 배열: 시간 다 쓴 task는 expired로 — 훑지 않는다

active가 텅 비면

포인터 두 개를 맞바꾸는 '한 번'의 교체 — 새 epoch 시작.

결과

task 1,000개여도 선택 연산 = 2회 (find-first-bit + 큐 머리 꺼내기)

실습 4

O(n) vs O(1) — 카운터로 비교

할 일

① 두 도구에서 task를 8개 이상으로 늘리기
② '비교/연산 카운터'의 차이를 관찰·캡처
③ 워크시트 S4의 답 5개 채우기

결과물

워크시트 S4 완료 + 카운터 비교 캡처(S8에서 사용)

▶ O(n) 스케줄러 | ▶ O(1) 스케줄러 | 📋 Day 2 워크시트

확인 「자유 모드에서 task 8개 이상, 두 도구의 카운터 차이를 캡처해 둔 조는 손!」

이 시간의 도착점

「가장 적게 받은 사람이 다음 주자」 규칙 하나로 차례·우선순위·
반응성이 전부 따라 나오는 CFS의 구조를 설명한다

예측 — O(1)도 빠른데 왜 갈아치웠을까요? '빠름' 말고 부족했던 것은?

CFS — 시계 하나로 공평을 산다

vruntime 경주

- task마다 '받은 시간' 시계 vruntime
- 다음 주자 = vruntime 최소(맨 왼쪽)
- 달리면 시계가 오르고, 뒤로 밀린다
- 규칙 끝. 나머지는 전부 따라 나온다

우선순위와 반응성

- nice = 시계 '속도' — 중요하면 천천히 오름
- → 같은 실시간에 시계가 덜 올라 더 자주 뽑힘
- 잠꾸러기 기상 → min 근처로 보정
- (과거 몰아받기 방지 + 즉시 반응)

맨 왼쪽을 빨리 찾는 나무 — 레드블랙 트리

왜 트리인가

vruntime 순으로 늘 정렬돼 있어야 하고, 삽입·삭제·최솟값이 전부 빨라야 한다
→ 스스로 균형 잡는 이진 탐색 트리(RB Tree), 모든 연산 O(log n)

균형의 도구 둘

색 바꾸기(싸다, 위로 미룸) + 회전(그 자리에서 해결)

규칙 맛보기

빨간 노드의 자식은 빨강일 수 없다 — 이 금지 하나가 기울기를 막는다.

실습 5

CFS — 공평의 실감

할 일

① CFS 도구: nice −5 task의 '받은 실시간 2배' 만들기
② 잠꾸러기 기상 보정 관찰
③ 워크시트 S5의 답 5개 (심화 2문항 포함)

결과물

워크시트 S5 완료 + 실시간 2배 화면

▶ CFS 스케줄러 | ▶ 심화: 레드블랙 트리 | 📋 Day 2 워크시트

확인 「nice −5 task의 받은 실시간이 2배가 되는 화면을 만든 조는 손 들어 주세요.」

이 시간의 도착점

**쪼개서 주고(split) 짝 맞춰 합치는(merge) buddy와,
페이지를 재단해 파는 slab의 2층 협업을 설명한다**

예측 — 20K를 달라고 하면 OS는 정확히 20K를 줄까요?

buddy — 반으로 쪼개고, XOR로 짝을 찾는다

할당

- 2의 거듭제곱 블록만 취급
- 20K 요청 → 32K 지급 (내부 단편화)
- 큰 블록을 반씩 쪼개 내려간다
- free list가 크기별 재고 장부

해제와 병합

- buddy 주소 = 내 주소 XOR 내 크기
- 계산 한 번 — 검색이 없다
- buddy가 free면 병합, 또 그 위로 연쇄
- 반납 순서가 병합을 열고 닫는다

slab — buddy의 도매 페이지를 소매로 재단

2층 구조

buddy(도매상): 4KB 페이지 단위로만 거래 / slab(소매점): 페이지를 임대해 64B·128B… 물건으로 재단

진열대 셋

partial(부분 사용 — 새 할당 1순위) · full · empty(반납 후보)

효과

같은 크기 반복 할당·해제에서 낭비와 시간 모두 절약 — 커널 객체의 집.

buddy·slab — 배급 실습

할 일

① buddy 해부: 20K→32K, XOR 짝 찾기 관찰
② buddy 운용: 128K 통짜 복구(병합 3연쇄) 도전
③ slab: partial 1순위 확인 · 워크시트 S6 답 5개

결과물

워크시트 S6 완료 + 128K 복구 화면

▶ buddy 해부

▶ buddy 운용

▶ slab allocator

📋 Day 2 워크시트

확인 「128K 통짜 복구(병합 3연쇄)를 만든 조는 화면을 들어 보여 주세요.」

이 시간의 도착점

주소가 값이 아니라 「경로」임을 분해식으로 보이고, 유효하지 않은 주소와 모자란 RAM에 OS가 각각 어떻게 답하는지 말한다

예측 — 프로세스 A와 B가 '똑같은 주소'를 읽으면, 같은 데이터가 나올까요?

주소 = 경로 — 10 + 10 + 12의 산수

32bit 주소의 세 조각

[PGD 인덱스 10bit] [PTE 인덱스 10bit] [offset 12bit]
= 큰 지도 몇 칸 · 작은 지도 몇 칸 · 페이지 안 몇 걸음

왜 하필 10bit?

표도 4KB 페이지 1장: 4KB ÷ 엔트리 4B = 1024칸 = 10bit. 산수다.

walk와 TLB

표 2장 걷기 = RAM 접근 2회 추가 → TLB(지름길 캐시)가 0회로.

격리와 사고 — CR3, Page Fault, SIGSEGV

같은 주소, 다른 세계

- 프로세스마다 자기 PGD (CR3가 지정)
- 같은 가상 주소 → 다른 물리 frame
- 이것이 '격리' — 남의 Memory에 닿을 수 없다

두 가지 사고

- P=0 (아직 안 올라옴) → Page Fault → OS가 buddy에서 frame 받아 채우고 재시도
- 내 땅이 아예 아님(find_vma 미발견) → SIGSEGV — 프로그램 강제 종료

RAM이 모자라면 — 두 줄의 LRU와 second chance

리눅스의 근사 LRU

진짜 LRU(매 접근마다 목록 갱신)는 불가능 → MMU가 켜 주는 R비트 + active/inactive 두 줄

회수(kswapd)

inactive 꼬리부터: R=1 → 기회 한 번 더(머리로) / R=0 → 축출(dirty면 write-back 먼저)

오판의 청구서

쫓아낸 페이지를 다시 쓰면 refault — 디스크에서 재적재(수만 배 느림).

실습 7

가상 메모리 — 지도·문지기·회수

할 일

① 2단 도구: 10+10+12 분해와 walk
② VMA 도구: find_vma 미발견 → SIGSEGV 화면 만들기
③ LRU 도구: 압박 → second chance 관찰 · 워크시트 S7 답 6개

결과물

워크시트 S7 완료 + SIGSEGV 화면

▶ x86 2단 (PGD·PTE)

▶ 주소 변환·TLB·#PF

▶ VMA 에뮬레이터

▶ LRU 페이지 회수

▶ 심화: interval tree

📋 Day 2 워크시트

확인 「SIGSEGV 화면을 만든 조는 손! 그 주소가 왜 '남의 땅도 아닌 빈 땅'인지 한 문장으로.」

이 시간의 도착점

오늘의 도착점 그 자체 — Terminal의 ls ⏎ 한 줄이 처리되는
전 과정을 여섯 단계로 빈칸 없이 이어 말한다

오늘 배운 모든 세션이 이 한 문제에 한 번씩 등장합니다.

명령 한 줄의 여행 — ls ⏎ 부터 Prompt까지

① bash가 자신을 복제

fork()

② 자식이 ls로 변신

execve()

③ CPU 차례 배정

스케줄러(CFS)

④ 주소를 표로 변환

MMU (PGD→PTE)

⑤ 첫 접근 페이지 적재

#PF + buddy

⑥ 좀비 회수, Prompt 복귀

wait()

여섯 칸 = 워크시트 S8의 여섯 빈칸

실습 8

명령 한 줄의 여행 — 완주

할 일

① 워크시트 S8: 여섯 빈칸 채우기
② 짝에게 1분 설명 → 체크
③ 이름 확인 → 채점하기 → 완료 화면 캡처

결과물

채점 완료 화면(이름 포함) — [강사 확정: 제출 채널]로 제출

▶ 도구 모음 (index)

📋 Day 2 워크시트

확인 「채점하기를 누르면 완료 화면이 열립니다. 점수 화면을 캡처해 제출해 주세요.」

오늘의 마침표

여러분은 이제 '뒤편'을 아는 사용자입니다

오늘 증명한 것

ls ⏎ 한 줄 = fork → execve → CFS → MMU → #PF+buddy → wait
— 여섯 단계를 여러분의 입으로 이어 말했습니다.

다음 시간 (Day 3)

명령 여러 줄을 파일 하나로 — Script 자동화

오늘 이해한 '한 줄의 뒤편' 위에, 내일은 그 줄들을 묶어 봇처럼 부립니다.